Duron Postgres Adapter: Storage Architecture Analysis

Context

This document summarizes a technical analysis of the Postgres adapter in `geut/duron` (a type-safe job queue system for Node.js and Bun), prompted by a tweet describing how traditional Postgres-backed queues (PGMQ, River, Que, pg-boss) suffer from MVCC bloat due to UPDATE/DELETE-heavy patterns, versus PgQ's approach using rotating tables with TRUNCATE.

The goal: evaluate whether duron's current design is vulnerable to the same problem, and what architectural changes would mitigate it.

The Underlying Problem: Dead Tuples and MVCC

How Postgres actually handles UPDATE/DELETE

Postgres never modifies rows in place. Every `UPDATE` creates a new row version and marks the old one as a "dead tuple". `DELETE` just marks the row as dead. The old versions stay on disk until cleanup.

This is **MVCC (Multi-Version Concurrency Control)**: readers and writers don't block each other because multiple versions of each row coexist. Each row has hidden `xmin` (creating transaction) and `xmax` (invalidating transaction) fields, and each transaction sees the version consistent with its snapshot.

Autovacuum and the xmin horizon

Dead tuples are eventually reclaimed by **autovacuum**. But autovacuum can only clean a dead tuple if no live transaction might still need it.

The **xmin horizon** is the oldest active transaction ID. Autovacuum cannot clean any dead tuple newer than this horizon.

Why idle-in-transaction is catastrophic

A session that runs `BEGIN` but never commits (due to a bug, a hung worker, a misbehaving pool) with a real XID holds the xmin horizon frozen. As long as that session is alive, autovacuum cannot reclaim anything generated after it started — **across the entire database**, not just the tables that session touched.

The tweet's scenario: a 6-minute idle-in-tx session on a high-throughput queue generates millions of unreclaimable dead tuples, leading to table bloat and degraded performance.

Why TRUNCATE is different

`TRUNCATE` deletes the underlying file physically. It generates no dead tuples, is instantaneous regardless of row count, and doesn't interact with the xmin horizon. `DROP TABLE` on a partition has the same property.

Monitoring queries

```
-- Dead tuples per table
SELECT schemaname, relname, n_live_tup, n_dead_tup,
       round(n_dead_tup::numeric / nullif(n_live_tup, 0) * 100, 2) AS dead_pct
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC;

-- Idle-in-transaction sessions
SELECT pid, now() - xact_start AS duration, state, query
FROM pg_stat_activity
WHERE state = 'idle in transaction'
ORDER BY xact_start;

-- Current xmin horizon holders
SELECT backend_xmin FROM pg_stat_activity
WHERE backend_xmin IS NOT NULL
ORDER BY age(backend_xmin) DESC LIMIT 5;
```

Current State of Duron's Postgres Adapter

Schema (in `src/adapters/postgres/schema.ts`)

Three tables:

- `jobs` — with mutable `status`, `updated_at`, timestamps, and ~15 indexes
- `job_steps` — with mutable `status`, `retries_count`, `history_failed_attempts`,

cascade delete from `jobs`

- `spans` — OpenTelemetry spans with FKs to jobs and steps

Write patterns (in `src/adapters/postgres/base.ts`)

- ~26 UPDATE/DELETE operations
- Job lifecycle: `INSERT (created)` → `UPDATE (active)` → `UPDATE (completed|failed)` = minimum 2 dead tuples per job in `jobs`
- `job_steps` worse: each retry updates status, retries_count, history_failed_attempts, updated_at
- Hot path uses `UPDATE ... SET status = active` with `FOR UPDATE SKIP LOCKED` via CTE

What duron does RIGHT

- **No explicit transactions wrapping the job handler.** The adapter uses atomic single-query CTEs. The worker holds no Postgres transaction while running user code.
- Transactions are kept as short as possible — just the time needed to fetch/claim/update state.
- `SKIP LOCKED` is used correctly to avoid worker contention.

What duron does NOT have

- **No automatic retention.** `_deleteJob` and `_deleteJobs` exist but must be called manually. Completed jobs accumulate indefinitely.
- All jobs (live, completed, failed) share the same table. The hot path's indexes must scan through all historical entries.

Revised assessment of the tweet's relevance to duron

The tweet's **catastrophic scenario** (6-min idle-in-tx blocking autovacuum) does NOT apply to duron's adapter directly — the adapter is well-designed in this respect. It could still happen if the user shares the Postgres database with other parts of their application that misbehave.

The **baseline problem** (UPDATE-heavy patterns creating constant pressure on autovacuum) DOES apply. At low throughput this is invisible. At high sustained throughput (thousands of jobs/sec), autovacuum runs constantly and index bloat accumulates because:

1. Completed jobs are LIVE tuples, not dead — vacuum doesn't remove them. The table

grows forever without retention.

2. Each vacuum must scan the entire table, including millions of irrelevant completed jobs, just to reclaim a small number of dead tuples generated by live job updates.
3. Many indexes (jobs has ~15) all need maintenance on every update.

Design Options Considered

Option 1: Table per state

Split into `jobs_created`, `jobs_active`, `jobs_completed`, `jobs_failed`. Each state transition is a `DELETE ... RETURNING + INSERT`.

Pros: Hot tables stay small. **Cons:**

- A job with 3 retries moves between tables 7+ times
- Foreign keys become impossible or ugly (which table does `job_steps.job_id` point to?)
- Multi-table transactions needed for every state change
- Large code complexity increase

Verdict: Elegant in concept, too expensive in practice.

Option 2: Partitioning alone (by time)

Keep one logical `jobs` table, partitioned by `created_at` (e.g., daily). Retention via `DROP TABLE jobs_2026_03_15`.

Pros:

- Retention by DROP (no dead tuples, instant)
- Partition pruning on time-range queries
- Code almost unchanged

Cons:

- The current day's partition still contains mixed live/completed jobs
- Still generates UPDATE pressure on the active partition
- Hot partition is still hot

Option 3: Active/Archive split (chosen direction)

Split the schema:

- `jobs_active` + `job_steps_active` — ALL live jobs (created and active state)
- `jobs_archive` + `job_steps_archive` — all terminated jobs (completed, failed, cancelled)

Lifecycle:

1. `INSERT` into `jobs_active` on creation
2. All UPDATEs (status transitions, retries) happen in `jobs_active`
3. On terminal state: single transaction that `DELETE ... RETURNING` s from active and `INSERT` s into archive
4. Retention runs on archive only

Why this beats the other options:

- `jobs_active` size proportional to in-flight work, NOT to historical volume. Always small.
- Vacuum on `jobs_active` is microseconds, not minutes
- Hot path indexes stay small and fit in memory
- Only ONE move per job (at termination), not one per state transition
- Code changes minimal compared to the table-per-state design
- Archive receives almost pure INSERTs — minimal dead tuple generation of its own

Relation to Dead Letter Queues

The active/archive split is a **storage/performance** concern. A DLQ is a **semantic/operational** concern (what happens to messages that fail terminally, so a human can inspect them).

They're orthogonal. Duron's current `status = 'failed'` effectively serves as a logical DLQ — failed jobs remain visible and queryable. This can coexist with active/archive: jobs are split by "alive vs terminated", and within the archive, `status` still distinguishes success from failure.

Partitioning Decision

The question

Should `jobs_archive` be partitioned by day (daily `DROP TABLE` for retention), or kept as a single table?

The constraint

No scripts required for duron to function correctly. A retention cron that drops old partitions is acceptable (if it fails one day, nothing breaks). A script required for the system to operate correctly is NOT acceptable.

The problem with time-range partitioning

Postgres does NOT auto-create partitions. If an `INSERT` arrives with a `finished_at` matching no existing partition, the INSERT fails. This means time-range partitioning requires a **critical** script that creates future partitions ahead of time. This violates the constraint.

Mitigations evaluated

- **DEFAULT partition as safety net:** Catches INSERTs that don't match any explicit partition. Downgrades the creation script from "critical" to "important". Works, but the DEFAULT partition accumulates and loses the partitioning benefit.
- **Create many partitions in advance:** Run the creation script monthly, create 90 days of future partitions. Tolerates long script failures but still requires the script.
- **pg_partman:** Postgres extension that handles partition management. Requires installation on the database server, not available on all managed Postgres providers. Breaks the "works on vanilla Postgres" promise.
- **Hash partitioning:** Creates partitions once at setup, never again. But loses the ability to drop old partitions by time — defeats the main point.
- **No partitioning:** Accept that retention = `DELETE` with dead tuples, accept it as an admin operation.

Decision

Go with active/archive split WITHOUT partitioning the archive.

Rationale:

- The archive tables receive almost exclusively INSERTs. Their natural bloat is minimal.
- Retention is a periodic admin operation, not a hot-path concern.
- `DELETE ... WHERE finished_at < X LIMIT batch_size` in a loop is manageable.

- No critical scripts required.
- The user's cron can run `pruneArchive()` on any schedule; if it fails, the archive just grows until next run.
- Users with extreme scale can partition `jobs_archive` themselves without duron's involvement, provided the schema is partitionable-friendly (no UNIQUE constraints that exclude the partition key).

Proposed Implementation

Schema changes

Replace the current `jobs` and `job_steps` tables with:

`jobs_active` — Same schema as current `jobs`, but contains only non-terminal jobs (status IN `created`, `active`). Keeps all current indexes needed for hot-path queries.

`job_steps_active` — Same schema as current `job_steps`. FK to `jobs_active.id` with `ON DELETE CASCADE`.

`jobs_archive` — Same schema as `jobs_active` plus no FK constraints from external tables. Fewer indexes — optimize for lookup by id, group_key, action_name; skip indexes that served hot-path queries.

`job_steps_archive` — Same schema as `job_steps_active` PLUS a denormalized `job_finished_at` column (copied from parent job at archival time). No FK. Minimal indexes.

`spans` — Keep as single table OR split into `spans_active` / `spans_archive` parallel to jobs. Simpler choice: keep single table, manage retention independently.

Design constraint: ensure the archive schema would permit hash or range partitioning if a user wants to add it without modifying duron. Any UNIQUE constraint on the archive should include a column that could serve as a partition key.

Code changes in the adapter

Creation path — INSERT to `jobs_active`, unchanged except for table name.

Update path (retries, status to active) — UPDATE `jobs_active`, unchanged except for table name.

Termination path — New transaction:

```
WITH moved_job AS (
  DELETE FROM jobs_active WHERE id = $1 RETURNING *
),
moved_steps AS (
  DELETE FROM job_steps_active WHERE job_id = $1 RETURNING *
),
inserted_job AS (
  INSERT INTO jobs_archive
  SELECT * FROM moved_job
  RETURNING finished_at
)
INSERT INTO job_steps_archive
SELECT ms.*, ij.finished_at AS job_finished_at
FROM moved_steps ms, inserted_job ij;
```

`getJob(id)` — Query `jobs_active` first. On miss, query `jobs_archive`. Cache the "likely location" if calling repeatedly on the same ID is common.

`getJobs(filters)` — Route based on filters:

- If `status IN ('created', 'active')` only, query `jobs_active` only
- If `status IN ('completed', 'failed', 'cancelled')` only, query `jobs_archive` only
- If mixed or no status filter, `UNION ALL` between the two
- Time-range filters on `finished_at` should bias to archive

Dashboard queries — May need two endpoints: "live jobs" and "historical jobs". Avoid the UNION ALL when possible.

New public method

```
await queue.pruneArchive({
  olderThan: '30d',        // or Date, or ms
  batchSize: 10000,        // optional, default reasonable
  maxBatches: 100,         // optional safety limit
})
```

Internally: loops `DELETE FROM jobs_archive WHERE finished_at < $threshold LIMIT $batchSize RETURNING id` and then deletes corresponding steps. Returns count of deleted jobs.

Alternative nuclear option:

```
await queue.truncateArchive()  // For users who want zero history
```

Documentation to add

A "Managing the archive" section that explains:

- Why the split exists (brief version of the MVCC problem)
- How to call `pruneArchive` from a cron
- Example with `setInterval` in a long-running app
- Note that if the user wants time-based partitioning, the archive schema supports it and they can add it themselves

---

What NOT to implement

- No internal cron or background worker inside duron
- No automatic partition creation or management
- No partition maintenance scripts shipped with the package
- No dependency on `pg_partman`, `pg_cron`, or other extensions
- No automatic retention — user must explicitly opt in by calling `pruneArchive`

Summary of Benefits

- Hot path operates on a small table regardless of historical volume
- Autovacuum on `jobs_active` completes in milliseconds
- Hot-path indexes remain small and cacheable in memory
- Archive grows linearly with throughput but doesn't affect live operations
- Retention is an explicit, bounded, admin operation the user controls
- No operational overhead introduced (no critical scripts, no dependencies)
- Users at extreme scale can add partitioning on top without duron changes
- Significant improvement over current design at scale, minimal complexity cost at small

scale

Tradeoffs Accepted

- Code complexity in the adapter increases (estimated 30-40% more LOC)
- Queries spanning live and historical jobs need `UNION ALL` or dual queries
- `getJob(id)` does up to 2 lookups instead of 1 (mitigated: active is tiny, miss is fast)
- Retention via `DELETE` generates dead tuples, but in a low-contention table that isn't on the hot path
- Migration path for existing duron users requires a one-off script (acceptable per user's decision)